03 – CSS

Styling the content of web pages

Jérôme Landré – **jerome.landre@ju.se**

OUTLINE

1) Introduction to CSS
2) History
3) CSS structure and syntax
4) CSS selectors
5) CSS properties
6) Interactions HTML/CSS
7) Flexbox and Grid
8) CSS frameworks
9) Best practices
10) Resources about CSS

1) INTRODUCTION TO CSS

- **Cascading Style Sheet (CSS)** is a language for writing style sheets, and is designed to describe the rendering of structured documents (such as HTML and XML) on a variety of media. CSS is used to describe the presentation of a source document, and usually does not change the underlying semantics expressed by its document language.

Images: https://www.freepnglogos.com/search.php?q=css

1) INTRODUCTION TO CSS

1) INTRODUCTION TO CSS

- Definition: Cascading Style Sheets level 3, the latest standard for styling web pages.
- Purpose: Separation of content (HTML) and presentation (CSS), enabling responsive, visually rich, and interactive web experiences.
- Key Features:
  - Advanced selectors
  - Animations and transitions
  - Flexbox and Grid layouts
  - Responsive design (media queries)
  - Custom fonts, shadows, gradients, and transformations

2) HISTORY

- Evolution of CSS:
  - CSS1 (1996): Basic styling (fonts, colors, margins).
  - CSS2 (1998): Positioning, z-index, media types.
  - CSS3 (1999–present): Modular approach, new selectors, animations, flexbox, grid, and more.
- Why CSS3?
  - Enhanced visual effects without images or scripts.
  - Improved performance and maintainability.
  - Support for responsive and mobile-first design.
  - Better browser compatibility and standardization.

3) CSS STRUCTURE AND SYNTAX

- The syntax of CSS is very simple:

```
selector {
  property: value;
}
```

- How to use CSS within HTML pages?
  - Internally (in the ".html" file)
    - style tag <style> … </style>
    - Inline in the tag itself using the style attribute

       …
  - Externally (in a ".css" file)

```
• Inline: <p style="color: blue;">
• Internal: <style>...</style> in <head>
• External: <link rel="stylesheet" href="styles.css"> (recommended)
```

4) CSS SELECTOR

- **Basic** Selectors:
  - element (e.g., p)
  - .class (e.g., .intro)
  - #id (e.g., #myheader)
  - * (universal selector)
  - selector1, selector2 (grouping)

…

- **Attribute** Selectors:
  - [attribute] (e.g., [type="text"])
  - [attribute^="value"] (starts with)
  - [attribute$="value"] (ends with)
  - [attribute*="value"] (contains)

…

4) CSS SELECTOR

- **Pseudo-classes**:
  - :hover, :active, :focus, :nth-child(n)
- **Pseudo-elements**:
  - ::before, ::after, ::first-line, ::first-letter

5) CSS PROPERTIES

- **Box Model**
  - width, height, margin, padding, border, box-sizing
- **Text**
  - font-family, font-size, text-align, text-shadow, line-height
- **Colors**
  - color, background-color, opacity, rgba(), hsla()
- **Backgrounds**
  - background-image, background-size, background-position, linear-gradient()
- **Layout**
  - display, position, float, flex, grid, z-index

5) CSS PROPERTIES

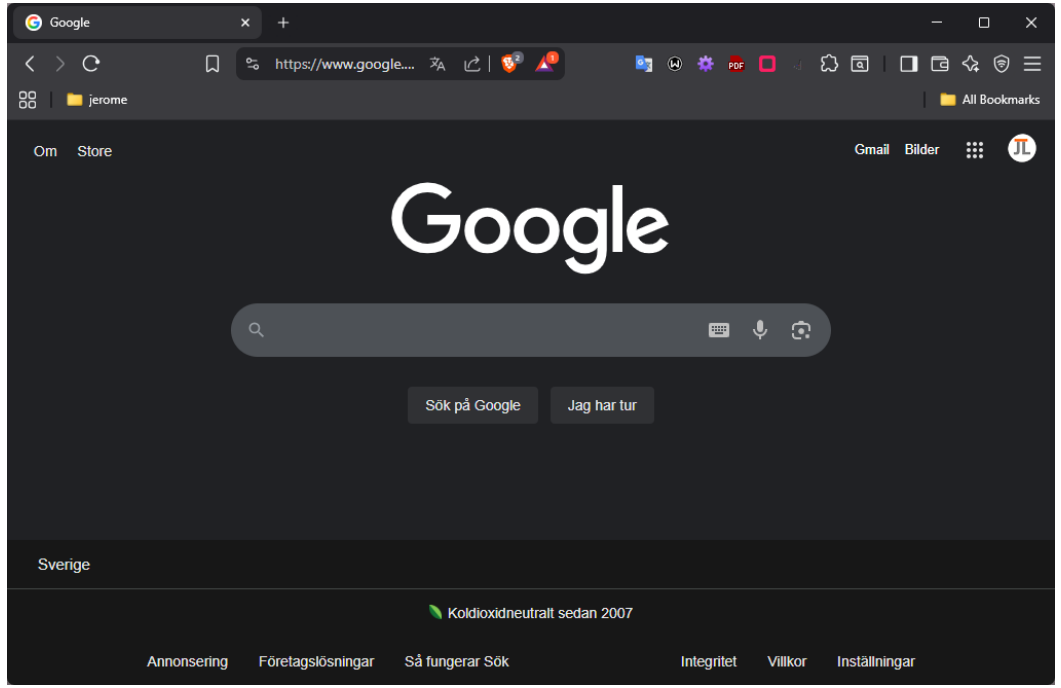
- **Transitions**
  - transition, transition-property, transition-duration, transition-timing-function
- **Animations**
  - @keyframes, animation, animation-duration, animation-delay
- **Transforms**
  - transform, rotate(), scale(), translate(), skew()

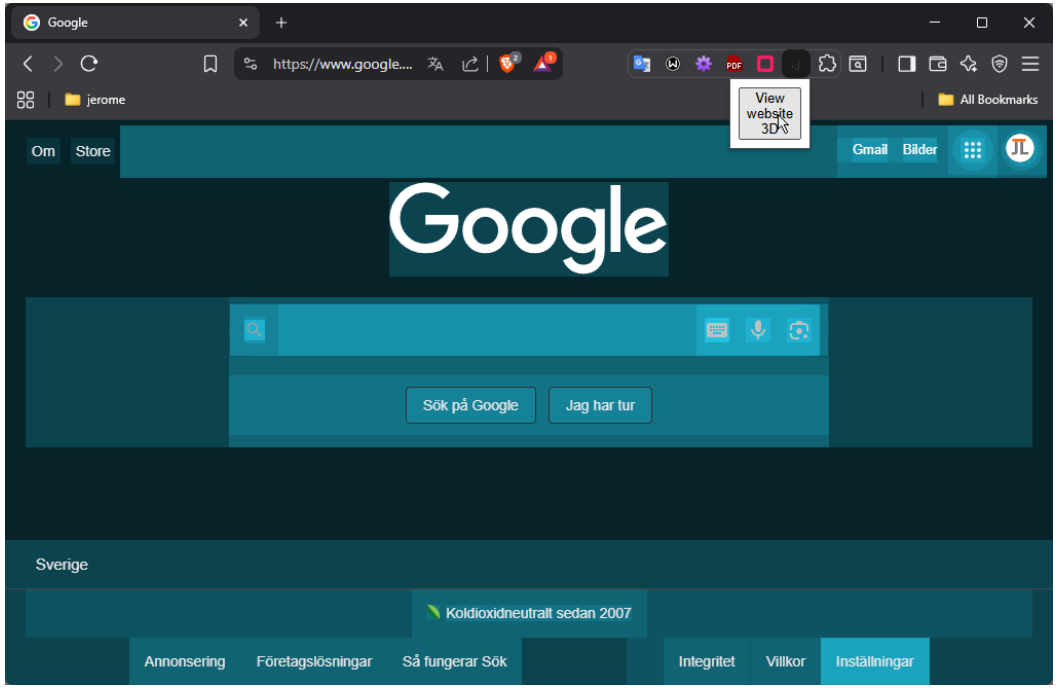
5) CSS PROPERTIES

- **Flexbox**
  - display: flex, flex-direction, justify-content, align-items
- **Grid**
  - display: grid, grid-template-columns, grid-gap, grid-area
- **Responsive**
  - @media, min-width, max-width

5) CSS PROPERTIES

- Any element on a web page is a **box**

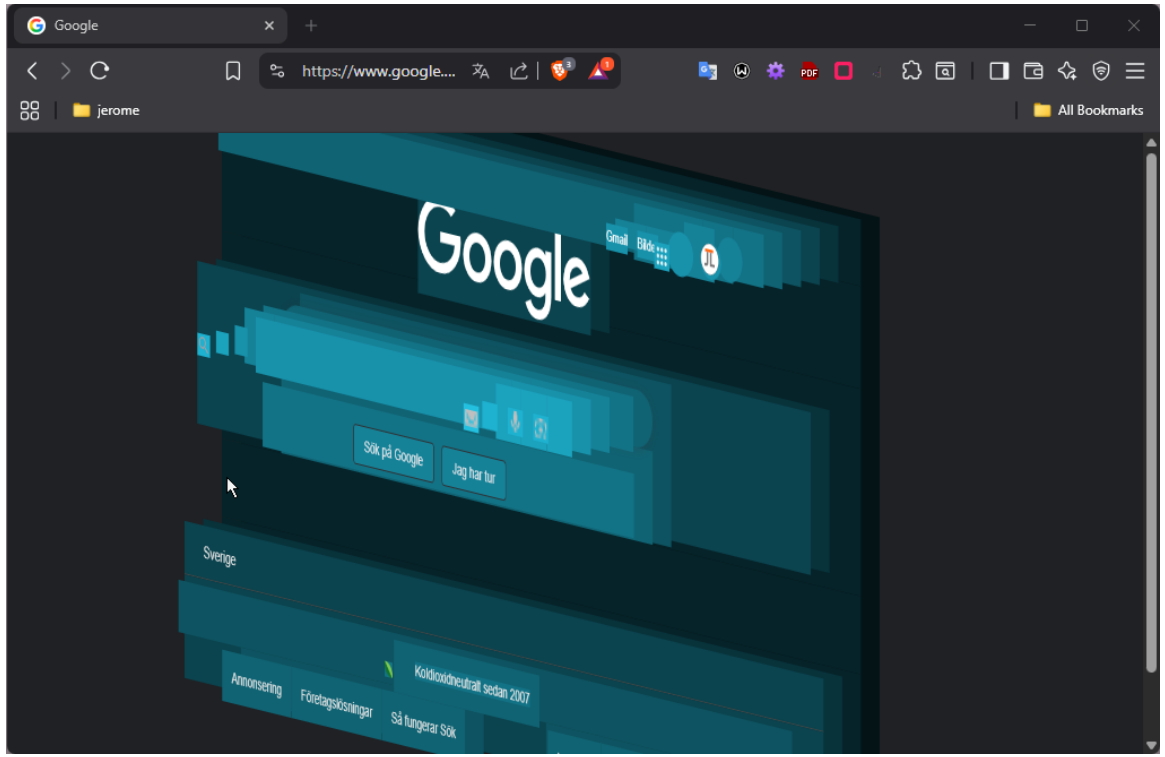


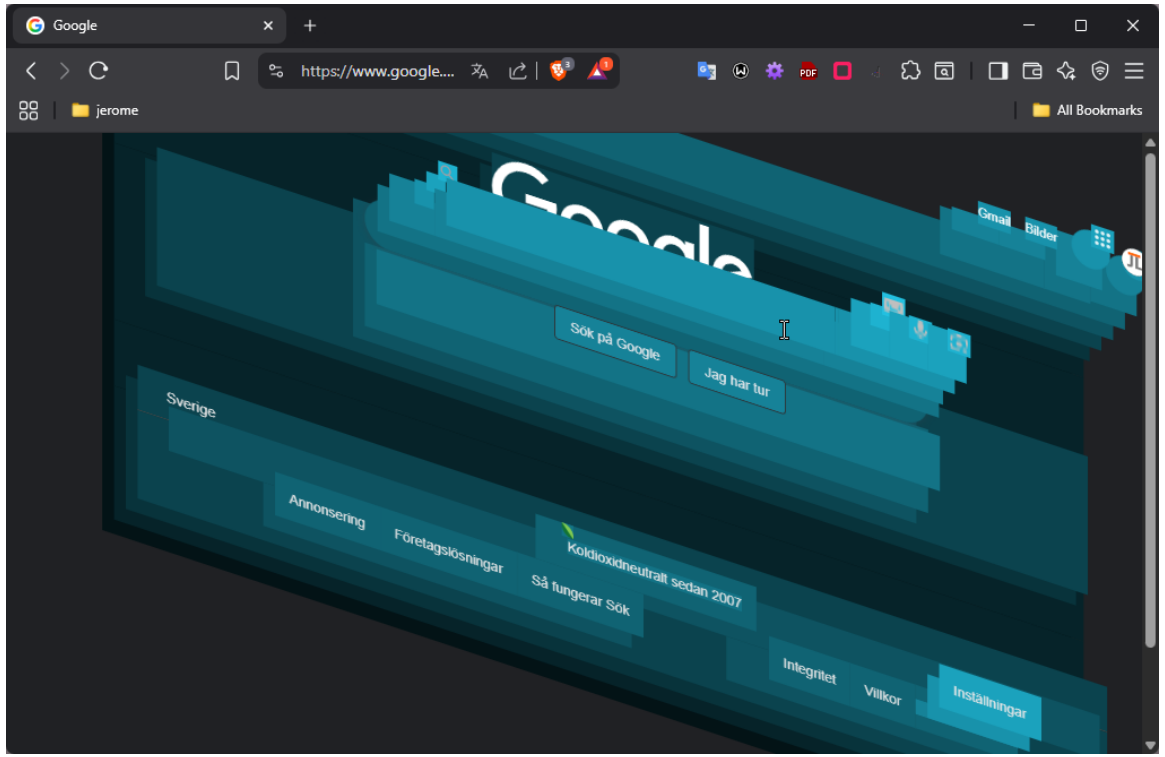


Source: 3D view website Chrome extension, https://chromewebstore.google.com/detail/3d-view-website/plodplkpcckemdhbahefdjbebnlngdih?hl=en

5) CSS PROPERTIES

- Any element on a web page is a **box**





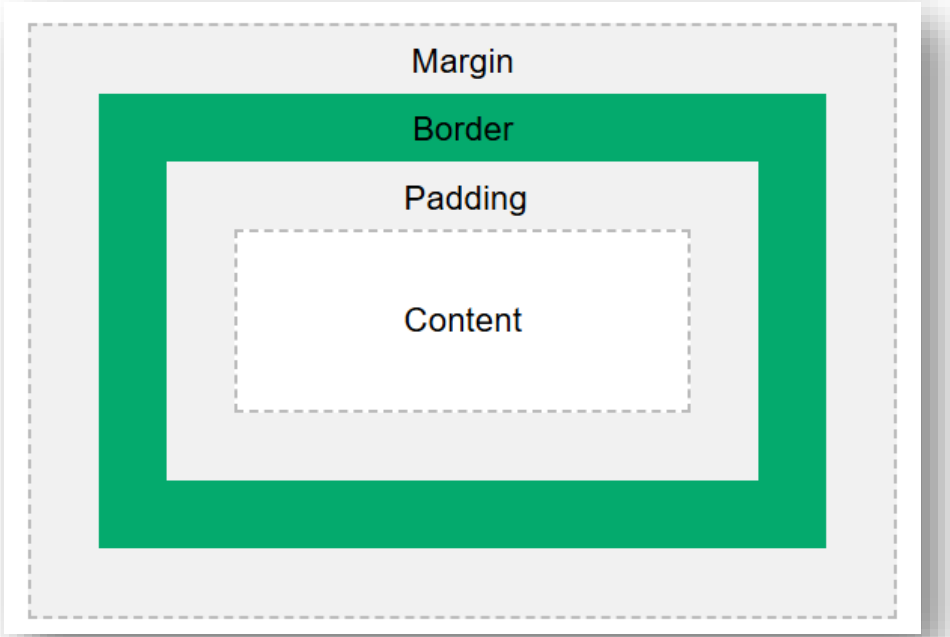
Source: 3D view website Chrome extension, https://chromewebstore.google.com/detail/3d-view-website/plodplkpcckemdhbahefdjbebnlngdih?hl=en

5) CSS PROPERTIES

- It means that building a website is dealing with **boxes**
- The **box model**:
  - Any element on the page is a box with a margin, a border, a padding and a content

Explanation of the different parts (from innermost part to outermost part):

- **Content** - The content of the box, where text and images appear
- **Padding** - Clears an area around the content. The padding is transparent
- **Border** - A border that goes around the padding and content
- **Margin** - Clears an area outside the border. The margin is transparent



Source: w3schools (2025), https://www.w3schools.com/css/css_boxmodel.asp

6) INTERACTIONS HTML/CSS

- **CSS** (Cascading Style Sheet) is a language used to assign properties to HTML tags to apply styles to elements
- A minimal HTML5 document and its external CSS style sheet:

index.html

project-languages > index.html > html

```
1   <!DOCTYPE html>
2   <html lang="en">
3   <head>
4       <meta charset="UTF-8">
5       <meta name="viewport" content="width=device-width, initial-scale=1.0">
6       <title>My page</title>
7       <link rel="stylesheet" href="styles-jl.css">
8   </head>
9   <body>
10      <p>My first paragraph...</p>
11  </body>
12  </html>
```

styles-jl.css

project-languages > styles-jl.css > ...

```
1   body {
2       font-family: sans-serif;
3   }
4
5   p {
6       color: darkgreen;
7   }
8
```

- Exercise:

Draw the **tree** of this HTML document and find where the styles apply.

index.html

project-languages > index.html > html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My page</title>
    <link rel="stylesheet" href="styles-jl.css">
</head>
<body>
    <p>My first paragraph...</p>
</body>
</html>
```

styles-jl.css

project-languages > # styles-jl.css > ...

```css
body {
    font-family: sans-serif;
}

p {
    color: darkgreen;
}

```

index.html

project-languages > index.html > html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My page</title>
    <link rel="stylesheet" href="styles-jl.css">
</head>
<body>
    <p>My first paragraph...</p>
</body>
</html>
```

styles-jl.css

project-languages > # styles-jl.css > ...

```css
body {
    font-family: sans-serif;
}

p {
    color: darkgreen;
}
```

Without the style file:

My first paragraph...

With the style file:

My first paragraph...

- Exercise:

Draw the **tree** of this HTML document and find where the styles apply.

Having the same stylesheet, can you guess where the style will apply?

index-v.1.html > ...

```html
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>My page</title>
        <link rel="stylesheet" href="styles-jl.css">
    </head>
    <body>
        <p>My first paragraph.</p>
        <div>A div in the middle.</div>
        <p>My second paragraph.</p>
    </body>
</html>
```

My first paragraph.

A div in the middle.

My second paragraph.

styles-jl.css > ...

```css
body {
    font-family: sans-serif;
}
p {
    color: darkgreen;
}
```

- Exercise:

Draw the **tree** of this HTML document and find where the styles apply.

Having the stylesheet version 2, can you guess where the style will apply?

index-v.2.html > ...

```html
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>My page</title>
        <link rel="stylesheet" href="styles-jl-v.2.css">
        <style>
            p { color: orange; }
        </style>
    </head>
    <body>
        <p>My <span class="important">first</span> paragraph.</p>
        <div>A <span class="important" id="veryimportant">div</span>
            in the middle.</div>
        <p style="color: magenta;">My <span class="important">
            second</span> paragraph.</p>
    </body>
</html>
```

styles-jl-v.2.css > ...

```css
body {
    font-family: sans-serif;
}
p {
    color: darkgreen;
}
.important {
    color: darkorange;
}
#veryimportant {
    color: red;
    font-weight: bold;
}
```

My first paragraph.

A **div** in the middle.

My second paragraph.

- Exercise:

Draw the **tree** of this HTML document and find where the styles apply.

Having the stylesheet version 2, can you guess where the style will apply?

index-v.3.html > ...

```html
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>My page</title>
        <style>
            p { color: orange; }
        </style>
        <link rel="stylesheet" href="styles-jl-v.3.css">
    </head>
    <body>
        <p>My <span class="important">first</span> paragraph.</p>
        <div>A <span class="important" id="veryimportant">div</span>
            in the middle.</div>
        <p style="color: magenta;">My <span class="important">
            second</span> paragraph.</p>
    </body>
</html>

```

styles-jl-v.2.css > ...

```css
body {
    font-family: sans-serif;
}
p {
    color: darkgreen;
}
.important {
    color: darkorange;
}
#veryimportant {
    color: red;
    font-weight: bold;
}
```

My first paragraph.

A **div** in the middle.

My second paragraph.

7) FLEXBOX AND GRID

- **Aligning elements on the page** can be tricky especially when the page becomes more and more complex (many elements displayed on the page)
- Old websites used a one-column model and a fixed-size resolution (the one of the screens: 640x480, 800x600, 1027x768…) to build websites
- You could use HTML tables to get a ”grid” layout for elements on your pages
- Then came the phones and the need for **responsive** web pages
- At this time, the 12-column 960px grid was used a lot.
- The floating elements using ”float:left;” or ”float: right;” added some degree of freedom for positioning
- If horizontal centering of elements was easy to get, vertical centering was difficult
- There was a **need** for easy **positioning** tags for HTML and CSS

7) FLEXBOX AND GRID

- **Flexbox** (Flexible Box Layout) in CSS is a layout model designed to efficiently arrange and align items within a container, enabling flexible and responsive designs. It allows items to adapt their size and position automatically to fill available space or shrink as needed, making layout management much simpler compared to older methods like floats or tables.
- Key features of flexbox:
  - **One-dimensional layout**: Flexbox is intended for layouts in a single direction (either rows or columns), not both at once like CSS Grid.
  - **Flex container and flex items**: By setting `display: flex` or `display: inline-flex` on a parent element (the flex container), all direct children become flex items and participate in the flex layout.
  - **Easy alignment**: Flexbox provides powerful alignment options along the main axis (set by `flex-direction`) and the cross axis, enabling vertical and horizontal centering, equal spacing, and reordering of items.
  - **Responsiveness**: Layouts built with flexbox automatically adapt as items grow, shrink, or wrap depending on screen size or content.

Source: Perplexity AI, https://www.perplexity.ai/

7) FLEXBOX AND GRID

- To use flexbox, set a parent container's CSS as follows:

```css
.container {
    display: flex;
    flex-direction: row; /* or column */
    justify-content: center; /* align items horizontally */
    align-items: center;     /* align items vertically */
}
```

- All direct children of `.container` will align according to these flexbox rules.
- Flexbox is widely supported and used for building navigation bars, image galleries, and other interface elements that need to adapt to different screen sizes and content lengths.

Source: Perplexity AI, https://www.perplexity.ai/

7) FLEXBOX AND GRID

index-v.4.html > ...

```html
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>My page</title>
        <style>
            p { color: orange; }
        </style>
        <link rel="stylesheet" href="styles-jl-v.4.css">
    </head>
    <body>
        <div class="container">
            <p>My <span class="important">first</span> paragraph.</p>
            <div>A <span class="important" id="veryimportant">div</span>
                in the middle.</div>
            <p style="color: magenta;">My <span class="important">
                second</span> paragraph.</p>
        </div>
    </body>
</html>
```

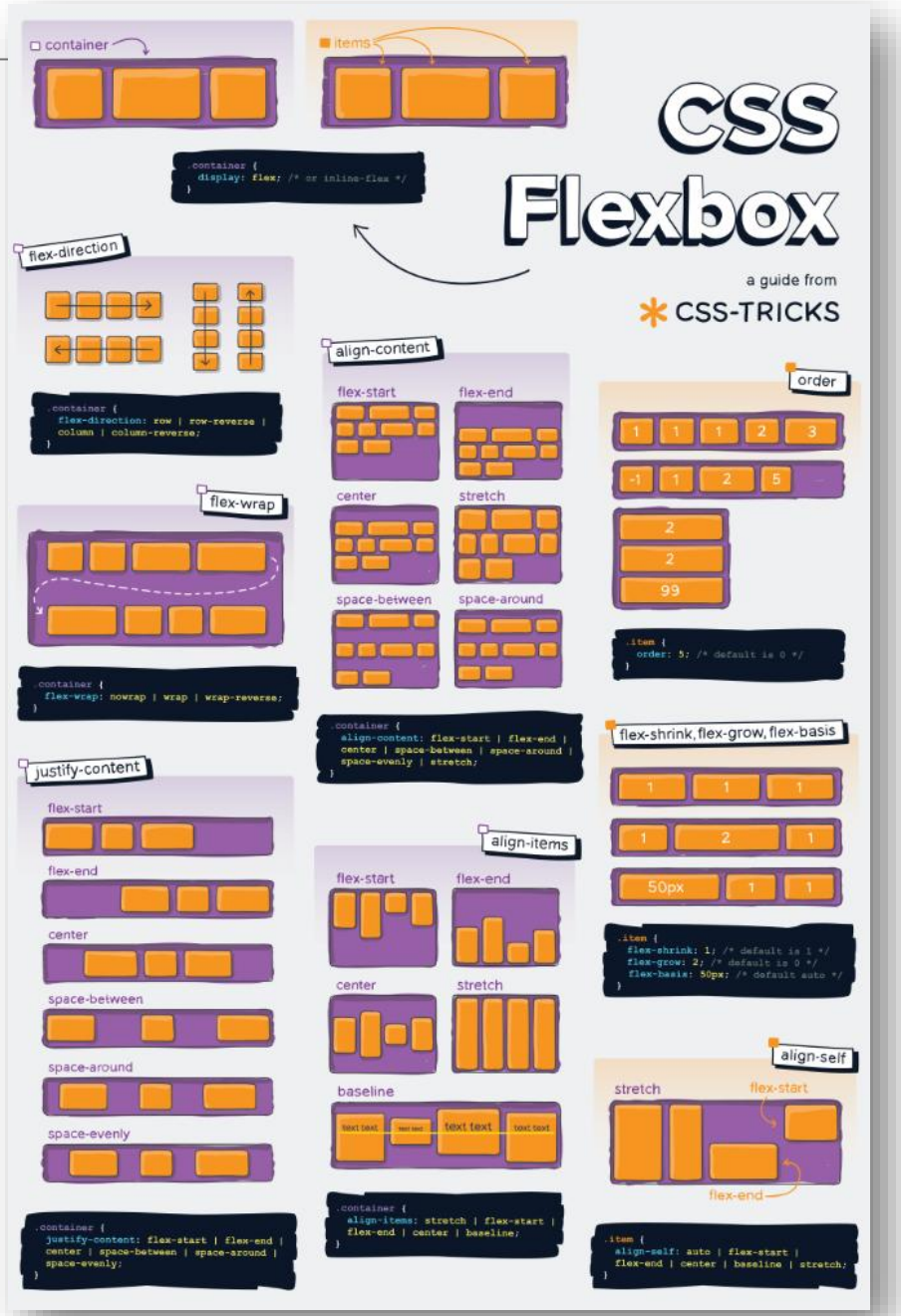
styles-jl-v.4.css > ...

```css
body {
    font-family: sans-serif;
}
.container {
    display: flex;
    flex-direction: row; /* or column */
    justify-content: center; /* align items horizontally */
    align-items: center;     /* align items vertically */
}
p {
    color: darkgreen;
}
.important {
    color: darkorange;
}
#veryimportant {
    color: red;
    font-weight: bold;
}
```

My first paragraph.A **div** in the middle.My second paragraph.

Source: Perplexity AI, https://www.perplexity.ai/

7) FLEXBOX AND GRID



Source: CSS tricks flexbox, https://css-tricks.com/wp-content/uploads/2022/02/css-flexbox-poster.png

7) FLEXBOX AND GRID

- CSS **Grid** is a **two-dimensional layout** system for the web that allows developers to organize content into both rows and columns, creating complex, responsive web layouts more easily and consistently. It works by defining a grid container with the CSS property display: grid (or inline-grid), which automatically treats its direct children as grid items. Developers can specify the size and number of rows and columns, control gaps between them, and place items precisely within the grid structure.
- CSS Grid's primary advantage is making layouts that **adapt well to different screen sizes** without relying on older layout hacks like floats or positioning. Unlike Flexbox, which is one-dimensional (rows or columns), CSS Grid is designed for two-dimensional layouts, offering finer control over spatial arrangement. It consists of components like grid container, grid items, grid lines, grid tracks (rows/columns), and grid areas.

Source: Perplexity AI, https://www.perplexity.ai/

7) FLEXBOX AND GRID

- Common CSS Grid properties include grid-template-columns, grid-template-rows, grid-gap, and alignment properties for controlling the layout and spacing of grid items.
- In summary, CSS Grid is a powerful modern CSS module that fundamentally changes web layout design by providing a flexible, intuitive way to build consistent two-dimensional layouts with rows and columns.

Source: Perplexity AI, https://www.perplexity.ai/ , CSS tricks Grid, https://css-tricks.com/wp-content/uploads/2022/02/css-grid-poster.png

8) CSS FRAMEWORKS

- If you want your CSS files to behave the same way on many different sites you build, you can use a **CSS Framework**.
- There are many of them on the web and their syntax and way to use is different from one framework to another:
  - Bootstrap
  - Bulma
  - Uikit
  - Tailwind CSS…

Source: GitHub page with CSS frameworks, https://github.com/troxler/awesome-css-frameworks

8) CSS FRAMEWORKS

- Example without Bootstrap:

boostrap-sans.html > ...

```
1   <!doctype html>
2   <html lang="en">
3     <head>
4       <meta charset="utf-8">
5       <meta name="viewport" content="width=device-width, initial-scale=1">
6       <title>Bootstrap demo</title>
7     </head>
8     <body>
9       <h1>Hello, world!</h1>
10    </body>
11  </html>
```

Hello, world!

Source: GitHub page with CSS frameworks, https://github.com/troxler/awesome-css-frameworks

8) CSS FRAMEWORKS

- Example with Bootstrap:

boostrap.html > ...

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Bootstrap demo</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
    rel="stylesheet" integrity="sha384-sRIl4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB"
    crossorigin="anonymous">
  </head>
  <body>
    <h1>Hello, world!</h1>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
    integrity="sha384-FKyoEForCGlyvwx9Hj09JcYn3nv7wiPVlz7YYwJrWVcXK/BmnVDxM+D2scQbITxI"
    crossorigin="anonymous"></script>
  </body>
</html>
```

Hello, world!

Source: GitHub page with CSS frameworks, https://github.com/troxler/awesome-css-frameworks

8) CSS FRAMEWORKS

- Example without Bootstrap with a form:

boostrap-sans-form.html > ...

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Bootstrap demo</title>
  </head>
  <body>
    <h1>Hello, world!</h1>
    <form>
        <div class="mb-3">
            <label for="exampleInputEmail1" class="form-label">Email address</label>
            <input type="email" class="form-control" id="exampleInputEmail1" aria-describedby="emailHelp">
            <div id="emailHelp" class="form-text">We'll never share your email with anyone else.</div>
        </div>
        <div class="mb-3">
            <label for="exampleInputPassword1" class="form-label">Password</label>
            <input type="password" class="form-control" id="exampleInputPassword1">
        </div>
        <div class="mb-3 form-check">
            <input type="checkbox" class="form-check-input" id="exampleCheck1">
            <label class="form-check-label" for="exampleCheck1">Check me out</label>
        </div>
        <button type="submit" class="btn btn-primary">Submit</button>
        </form>
  </body>
</html>
```

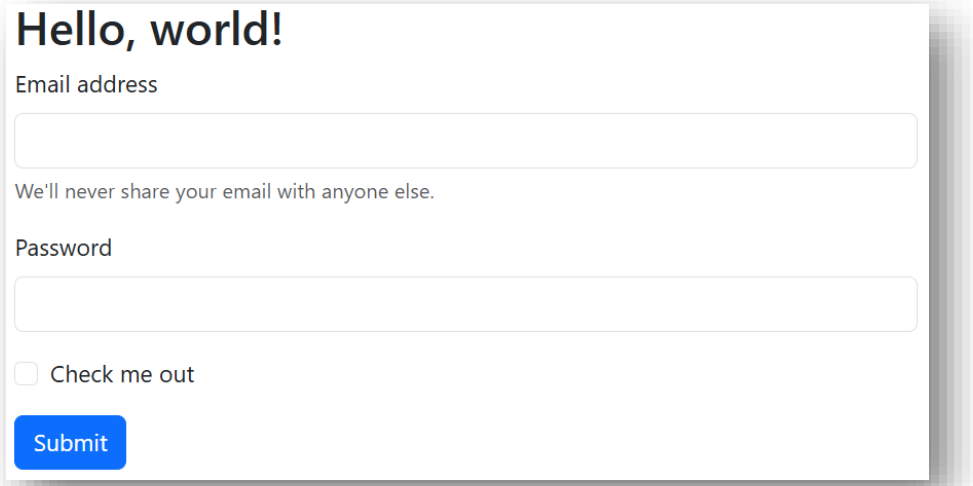
Source: https://getbootstrap.com/docs/5.3/forms/overview/

8) CSS FRAMEWORKS

- Example with Bootstrap:

boostrap-form.html > ...

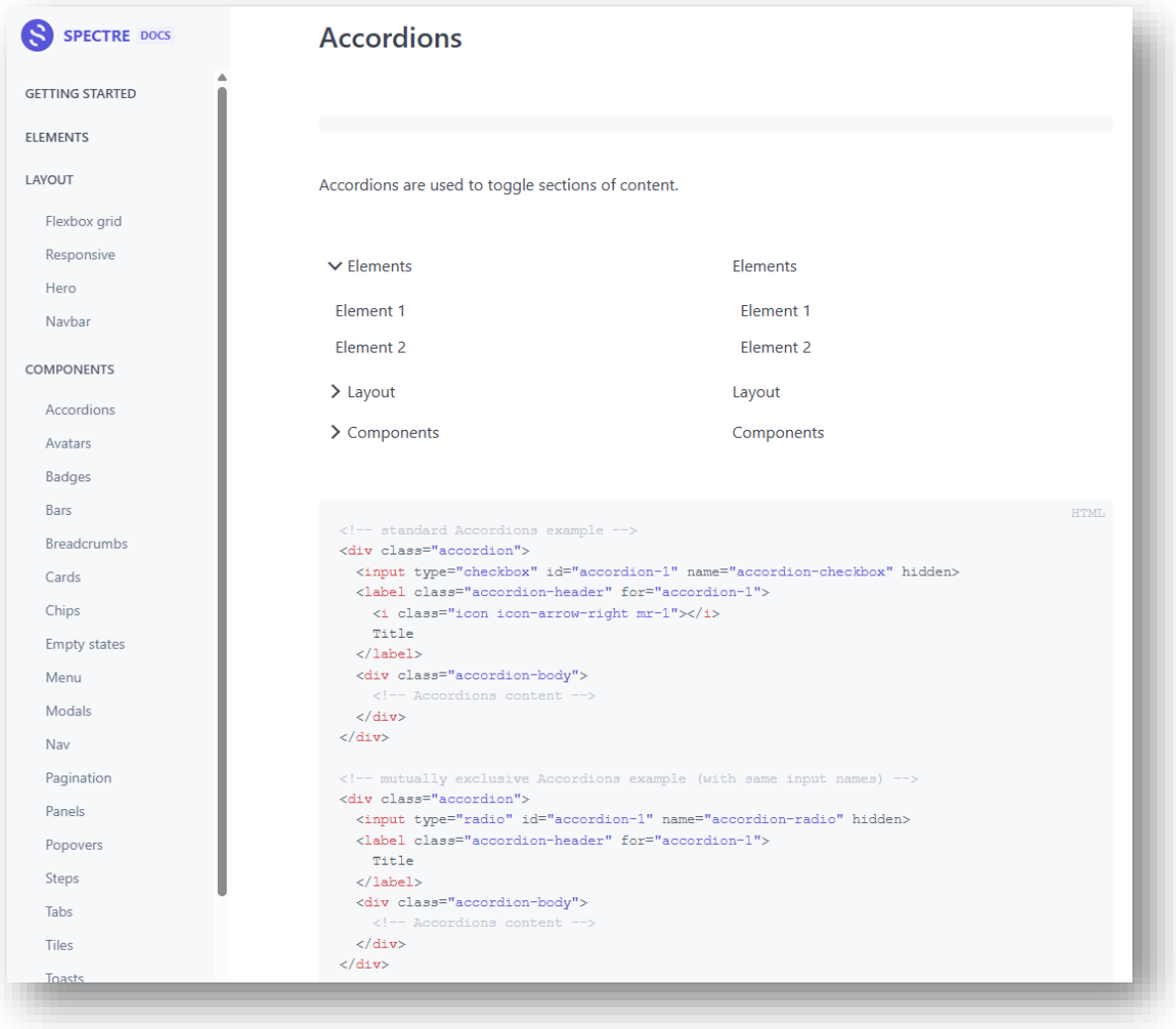
```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Bootstrap demo</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
    rel="stylesheet" integrity="sha384-sRIl4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB"
    crossorigin="anonymous">
  </head>
  <body>
    <h1>Hello, world!</h1>
    <form>
        <div class="mb-3">
            <label for="exampleInputEmail1" class="form-label">Email address</label>
            <input type="email" class="form-control" id="exampleInputEmail1" aria-describedby="emailHelp">
            <div id="emailHelp" class="form-text">We'll never share your email with anyone else.</div>
        </div>
        <div class="mb-3">
            <label for="exampleInputPassword1" class="form-label">Password</label>
            <input type="password" class="form-control" id="exampleInputPassword1">
        </div>
        <div class="mb-3 form-check">
            <input type="checkbox" class="form-check-input" id="exampleCheck1">
            <label class="form-check-label" for="exampleCheck1">Check me out</label>
        </div>
        <button type="submit" class="btn btn-primary">Submit</button>
    </form>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
    integrity="sha384-FKyoEForCGlyvwx9Hj09JcYn3nv7wiPVlz7YYwJrWVcXK/BmnVDxM+D2scQbITxI"
    crossorigin="anonymous"></script>
  </body>
</html>
```



Source: GitHub page with CSS frameworks, https://github.com/troxler/awesome-css-frameworks

8) CSS FRAMEWORKS

- A light-weight CSS framework is Spectre.css: https://picturepan2.github.io/spectre/
- The syntax is very simple and it is an easy-to-use tool



9) BEST PRACTICES

- Use **external** stylesheets for **maintainability**,
- Prefer **flexbox** (and grid) over floats for layout,
- Use **relative units** (%, vw, rem) for responsive design,
- Test on **multiple devices/browsers**.

10) RESOURCES ONLINE

- W3Schools CSS: https://www.w3schools.com/css/default.asp
- Mozilla Developer Network: https://developer.mozilla.org/en-US/docs/Web/CSS
- Flexbox: https://css-tricks.com/snippets/css/a-guide-to-flexbox/
- Grid: https://css-tricks.com/snippets/css/complete-guide-grid/
- Bootstrap: https://getbootstrap.com/docs
- Spectre: https://picturepan2.github.io/spectre/
- Can I use? (for browser support): https://caniuse.com/

REMEMBER:

WEB DEVELOPMENT FUNDAMENTALS

WEB DEV IS FUN

ANY QUESTIONS?